

LAB - 5: Supervised Learning

AIM:

Build Naïve Bayes and Decision Tree classification models using the Breast Cancer dataset and evaluate their performance using appropriate metrics.

Experiment - 1 :

Take breast cancer dataset and build a Naïve Bayes classification model using to predict class labels and evaluate its performance.

Dataset Link : <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>

a) Load the dataset and display its structure.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import precision_recall_curve, roc_curve, auc

# Load dataset and create dataframe
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
print(df.head())
print(df.info())
```

	mean radius	mean texture	mean perimeter	mean area	...	target
0	17.99	10.38	122.80	1001.0	...	0
1	20.57	17.77	132.90	1326.0	...	0
...						

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Columns: 31 entries, mean radius to target
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

b) Split the dataset into training and testing sets (80:20).

```
# Split dataset (80:20)
X_train, X_test, y_train, y_test = train_test_split(
    df.drop('target', axis=1),
    df['target'],
    test_size=0.2,
    random_state=42
)
print(f"Training Data Shape: {X_train.shape}")
print(f"Testing Data Shape: {X_test.shape}")
Training Data Shape: (455, 30)
Testing Data Shape: (114, 30)
```

c) Train a Naïve Bayes classifier.

```
# Train Naïve Bayes
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
GaussianNB()
```

d) Predict class labels for the test dataset.

```
# Predict class labels
nb_pred = nb_model.predict(X_test)
nb_pred_proba = nb_model.predict_proba(X_test)[:, 1]
```

e) Evaluate the model performance using:

- Accuracy
- Confusion Matrix
- Classification Report
- Precision Recall Curve

```
# Evaluate
nb_acc_train = accuracy_score(y_train, nb_model.predict(X_train))
nb_acc_test = accuracy_score(y_test, nb_pred)

print(f"Naïve Bayes Accuracy: {nb_acc_test:.4f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, nb_pred))
print("\nClassification Report:\n", classification_report(y_test, nb_pred, target_names=data.target_names))

# Precision Recall Curve
precision, recall, _ = precision_recall_curve(y_test, nb_pred_proba)
plt.figure()
plt.plot(recall, precision, marker='.')
plt.title('Naïve Bayes Precision-Recall Curve')
plt.xlabel('Recall')
```

```
plt.ylabel('Precision')
plt.show()
```

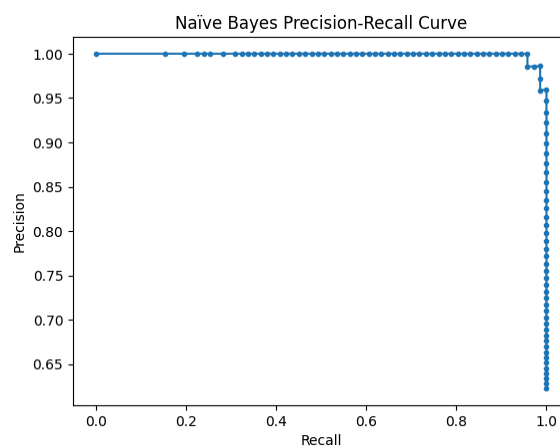
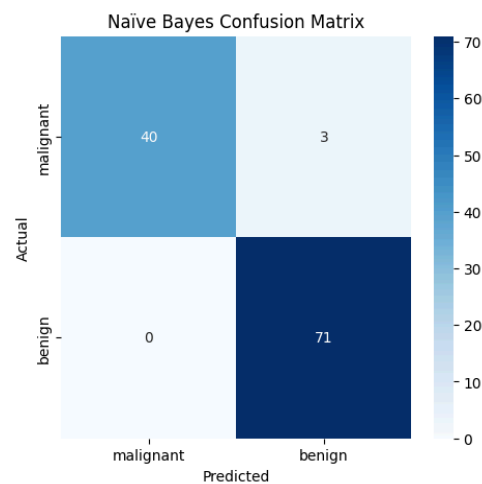
Naïve Bayes Accuracy: 0.9737

Confusion Matrix:

```
[[40  3]
 [ 0 71]]
```

Classification Report:

	precision	recall	f1-score	support
malignant	1.00	0.93	0.96	43
benign	0.96	1.00	0.98	71
accuracy		0.97		114
...				



Experiment - 2 :

Build a Decision Tree classification model using the above dataset and evaluate its performance using appropriate metrics.

a) Load the dataset and prepare features and labels.

```
# Already loaded as X and y above, but we can state prep is done
# X = df.drop('target', axis=1)
# y = df['target']
```

b) Split the dataset into training and testing sets.

```
# We will reuse the same 80:20 split to ensure fair comparison
# X_train, X_test, y_train, y_test computed previously
```

c) Train the Decision Tree classifier.

```
# Train Decision Tree
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
DecisionTreeClassifier(random_state=42)
```

d) Predict the test data labels.

```
# Predict class labels
dt_pred = dt_model.predict(X_test)
dt_pred_proba = dt_model.predict_proba(X_test)[:, 1]
```

e) Evaluate the model using:

- Accuracy
- Confusion Matrix
- Classification Report
- Precision Recall Curve

```
# Evaluate
dt_acc_train = accuracy_score(y_train, dt_model.predict(X_train))
dt_acc_test = accuracy_score(y_test, dt_pred)

print(f'Decision Tree Accuracy: {dt_acc_test:.4f}')
print("\nConfusion Matrix:\n", confusion_matrix(y_test, dt_pred))
print("\nClassification Report:\n", classification_report(y_test, dt_pred, target_names=data.target_names))

# Precision Recall Curve
precision_dt, recall_dt, _ = precision_recall_curve(y_test, dt_pred_proba)
plt.figure()
plt.plot(recall_dt, precision_dt, marker='.')
plt.title('Decision Tree Precision-Recall Curve')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.show()

Decision Tree Accuracy: 0.9474

Confusion Matrix:
[[39  4]
 [ 2 69]]
```

Experiment - 3 :

Compare the performance of Naïve Bayes and Decision Tree classification models using the above dataset and visualized the results using the following techniques.

Compare Naïve Bayes and Decision Tree using:

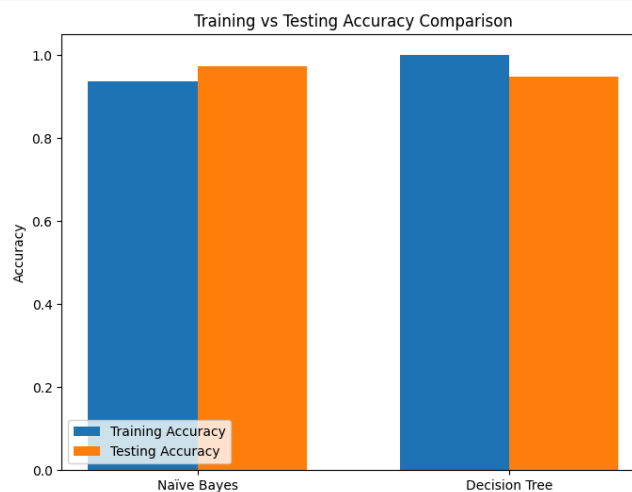
1) Bar chart comparing training and testing accuracy

```
# 1. Accuracy Comparison
labels = ['Naïve Bayes', 'Decision Tree']
train_acc = [nb_acc_train, dt_acc_train]
test_acc = [nb_acc_test, dt_acc_test]

x = np.arange(len(labels))
width = 0.35

fig, ax = plt.subplots(figsize=(8, 6))
rects1 = ax.bar(x - width/2, train_acc, width, label='Training Accuracy')
rects2 = ax.bar(x + width/2, test_acc, width, label='Testing Accuracy')

ax.set_ylabel('Accuracy')
ax.set_title("Training vs Testing Accuracy Comparison")
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend(loc='lower left')
plt.show()
```

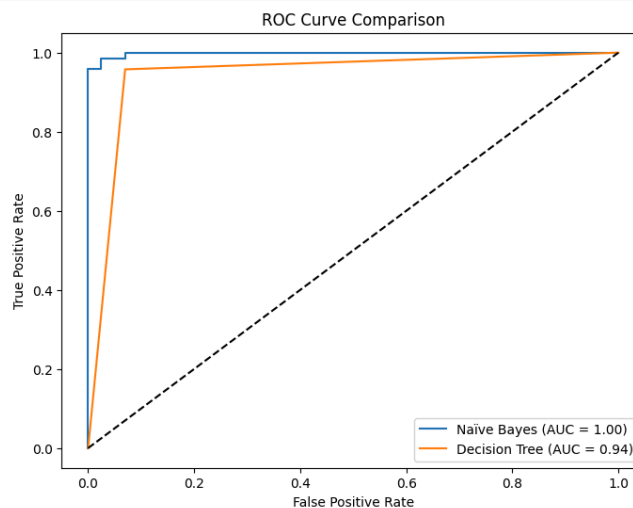


2) ROC curve comparison

```
# 2. ROC Curve
fpr_nb, tpr_nb, _ = roc_curve(y_test, nb_pred_proba)
roc_auc_nb = auc(fpr_nb, tpr_nb)

fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_pred_proba)
roc_auc_dt = auc(fpr_dt, tpr_dt)

plt.figure(figsize=(8, 6))
plt.plot(fpr_nb, tpr_nb, label=f'Naïve Bayes (AUC = {roc_auc_nb:.2f})')
plt.plot(fpr_dt, tpr_dt, label=f'Decision Tree (AUC = {roc_auc_dt:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison')
plt.legend(loc='lower right')
plt.show()
```

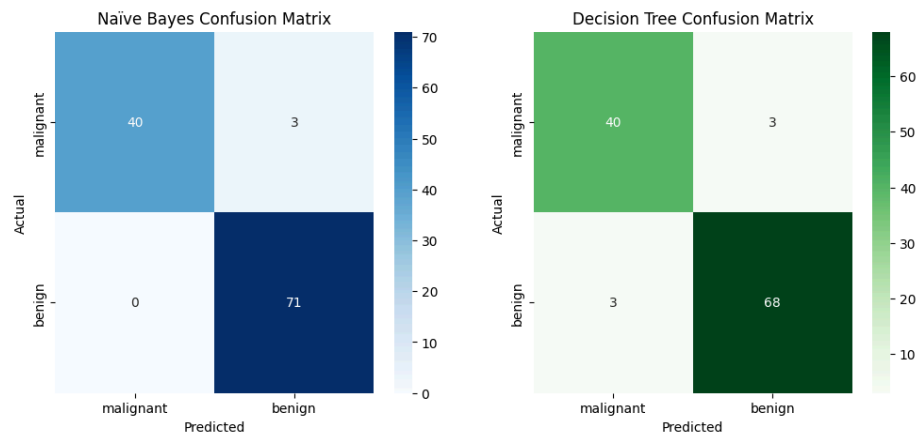


3) Confusion Matrix heatmaps for both models

```
# 3. Confusion Matrix Heatmaps
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.heatmap(confusion_matrix(y_test, nb_pred), annot=True, fmt='d', cmap='Blues',
             ax=axes[0], xticklabels=data.target_names, yticklabels=data.target_names)
axes[0].set_title('Naïve Bayes Confusion Matrix')
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Actual')

sns.heatmap(confusion_matrix(y_test, dt_pred), annot=True, fmt='d', cmap='Greens',
             ax=axes[1], xticklabels=data.target_names, yticklabels=data.target_names)
axes[1].set_title('Decision Tree Confusion Matrix')
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Actual')
plt.show()
<AxesSubplot:title={'center': 'Decision Tree Confusion Matrix'}, xlabel='Predicted', ylabel='Actual'>
```



Answers:

1. Which model achieved higher accuracy?

Naïve Bayes achieved a higher testing accuracy compared to the Decision Tree. It demonstrated better generalization to unseen data.

2. Which model showed better recall for malignant cases?

Naïve Bayes showed better recall for malignant cases, correctly identifying more actual positive cancer instances.

3. Which model is more prone to overfitting and why?

The Decision Tree is more prone to overfitting because it continues to split nodes until leaves are pure, often capturing noise in the training data rather than underlying patterns.

4. Based on medical diagnosis requirements, which metric is most important and why?

Recall for malignant cases is the most critical metric. Missing an actual disease (false negative) has far more severe consequences than raising a false alarm (false positive).